

Failure Recovery System Using Agentic AI in Software Development

A Multi-Agent Framework for Automated Program Repair

Meet Patel (23BCE177) Vedant Mehta (23BCE180)

School of Technology, Institute of Technology, Nirma University, Ahmedabad

Guide: Mr. AjayKumar Patel

Research Methodology and Seminar (RMS) — B.Tech CSE, Sem VI, AY 2025-26

Outline

- 1 Introduction
- 2 Related Work
- 3 System Architecture
- 4 Experimental Setup
- 5 Results
- 6 Discussion
- 7 Conclusion

Background & Motivation

- Modern CI/CD pipelines fail frequently due to compilation errors, runtime exceptions, and logical bugs.
- Manual debugging is slow, expertise-dependent, and expensive at scale.
- Classical Automated Program Repair (APR) — search-based, template-based — lacks reasoning and adaptability.
- LLMs can understand and generate code, but **single-step prompting** lacks structured reasoning and iterative validation.

Key Question

Can a *structured, multi-agent* LLM pipeline outperform a single-prompt LLM baseline at automated failure recovery — and does that answer depend on the model or the bugs?

- Traditional APR relies on fixed repair templates / limited search strategies.
- Single-step LLM repair lacks structured planning and iterative validation.
- Existing systems mostly target *patch generation*, not the full recovery workflow (detect → localize → diagnose → patch → validate).
- Little work evaluates multi-agent orchestration *across multiple benchmarks and multiple model capabilities* rather than one favorable setup.

Objectives

- ➊ Design a multi-agent failure-recovery architecture using Agentic AI.
- ➋ Automate detection, debugging, and validation via LLM-powered agents.
- ➌ Generalize to **three** benchmarks with heterogeneous test formats: QuixBugs, HumanEvalFix, DebugBench.
- ➍ Evaluate under **two** backends of very different capability: Gemini (hosted) vs. Qwen2.5-Coder 7B (local, on-device).
- ➎ Compare against a single-prompt baseline in *every* configuration.
- ➏ Measure success rate, Pass@1, MTTR — and document infrastructure issues found at scale.

Contributions

- A structured **six-agent** architecture for automated failure recovery.
- A **dataset-agnostic** execution layer unifying three test formats (tuple I/O, assert-based, LeetCode-style example I/O).
- A **local inference backend** (Qwen2.5-Coder 7B via Ollama) with multi-key quota-rotation for the Gemini backend.
- A full **3 datasets** $\times$ **2 backends** evaluation matrix — showing multi-agent's benefit is *conditional*, not fixed.
- Three documented **infrastructure defects** (threading/signals, timeout gaps, quota exhaustion) found only via multi-hour unattended runs.

Evolution of Program Repair Approaches

Approach	Reasoning	Iterative	Multi-Agent
GenProg	No	Yes	No
TBar	No	No	No
CoCoNuT	Learned	No	No
AlphaRepair	Learned	No	No
ChatRepair	LLM	Yes	No
SWE-Agent	LLM	Yes	No
Ours	LLM	Yes	Yes

Also related: SWE-bench, AutoCodeRover, Agentless, ReAct (agentic reasoning); autonomic computing's MAPE loop (self-healing systems).

Six-Agent Recovery Architecture

- ➊ **Failure Detection** (LLM) — run tests, classify failure type
- ➋ **Code Localization** (LLM) — pinpoint the faulty line
- ➌ **Debugging** (LLM) — root-cause analysis + fix strategy
- ➍ **Patch Generation** (LLM) — write corrected function
- ➎ **Patch Application** (Python) — apply & check syntax
- ➏ **Validation** (Python) — run full test suite

Coordinator manages the loop: retries up to 3 times, feeding validation errors back as context.

4 LLM calls/attempt $\times$ 3 attempts = up to **12 LLM calls per bug**.

Recovery Loop (Algorithm)

Require: Buggy program P , Test suite T , Max attempts $N = 3$

Ensure: Patched program P' or failure report

```
1:  $output \leftarrow \text{Execute}(P, T)$ 
2: for  $attempt = 1$  to  $N$  do
3:    $failure \leftarrow \text{FailureDetection}(P, output)$ 
4:    $location \leftarrow \text{CodeLocalization}(P, failure)$ 
5:    $analysis \leftarrow \text{Debugging}(P, failure, location)$ 
6:    $patch \leftarrow \text{PatchGeneration}(P, analysis)$ 
7:    $P' \leftarrow \text{PatchApplication}(P, patch)$ 
8:    $result \leftarrow \text{Validation}(P', T)$ 
9:   if  $result.all\_passed$  then
10:    return  $P'$ 
11:  else
12:     $output \leftarrow result.errors$  {feed back for retry}
13:  end if
14: end for
15: return Failure Report
```

Dataset-Agnostic Test Execution

Three benchmarks encode correctness differently:

- **Tuple I/O** (QuixBugs) — `[[inputs], expected]` applied to the function directly.
- **Assert-based** (HumanEvalFix) — a `check(candidate)` function runs as one pass/fail unit.
- **Example I/O** (DebugBench) — LeetCode-style `class Solution`, entry method invoked positionally.

A Problem abstraction (`test_style` tag) + executor dispatch lets `agents.py` / `coordinator.py` stay identical across benchmarks.

Dual LLM Backend with Quota-Aware Rotation

- Common interface: `generate(prompt) -> str`, two implementations selected at runtime.
- **Gemini backend** — hosted API, rate limiting, exponential backoff.
- **Local backend** — Qwen2.5-Coder 7B Instruct via Ollama, on-device (Apple Silicon GPU), no API cost.
- Free-tier Gemini quota can be exhausted mid-experiment (up to 12 calls/problem).
- Client rotates across **4 API keys** on detecting quota exhaustion (not transient errors) — verified to draw from independent quota pools.

Infrastructure Defects Found at Scale

- **Daemon-thread timeout leak:** original 5s timeout used a thread `join()` that never killed the thread — masked at small scale, but genuine infinite loops on larger benchmarks left CPU-bound threads accumulating and stalling the host.
Fix: SIGALRM-based timeout, armed/disarmed per test.
- **Unprotected exec path:** `HumanEvalFix`'s `check()` executes *at module scope inside* `exec()`, outside the timeout wrapper — a genuine infinite loop hung with no protection.
Fix: move the whole `exec()` call inside the timeout window.
- **Signal-in-thread bug:** a background-thread watchdog made every `signal.alarm` call raise `ValueError` (signals are main-thread-only), silently misreported as a 360s timeout.
Fix: keep the pipeline on the main thread; bound LLM calls via HTTP timeouts instead.

- **QuixBugs** — 40 classic algorithm programs, 1 single-line bug each; 31 with JSON tests used.
- **HumanEvalFix** — 164 HumanEval solutions with an implanted bug + full assert-based test suite each.
- **DebugBench** — 4,253 LeetCode-derived problems (C++/Java/Python); we use a fixed-seed, category-stratified sample of 119 Python problems (syntax, reference, logic, multiple-error categories).

Evaluation Matrix & Configuration

3 datasets × **2 backends**, multi-agent vs. single-prompt baseline in each cell.

Metrics

- Repair Success Rate
- Pass@1 / Pass@3
- Mean Time to Recovery (MTTR)
- Avg. Attempts

Config

- Temperature 0.2, max 2048 tokens
- Max 3 retry attempts / bug
- 5s test-execution timeout
- Apple M5 Pro, 24GB unified memory

QuixBugs / Gemini — Overall Results

Metric	Multi-Agent	Baseline
Bugs Fixed	24/31	23/31
Success Rate	77.4%	74.2%
Pass@1	74.2%	74.2%
Avg. MTTR (s)	81.1	5.6
Total API Calls	~140	31

Multi-agent leads by **+3.2 points**; the retry loop recovers levenshtein on attempt 2 after attempt 1 fails — baseline has no such mechanism.

Error-Type Breakdown (QuixBugs / Gemini)

Error Type	Bugs	Fixed	Rate
WrongOutput	21	16	76.2%
RuntimeError	3	3	100.0%
IndexError	3	3	100.0%
InfiniteLoop	2	1	50.0%
ValueError	1	1	100.0%

Clear-error-message bugs (Runtime/Index/Value) → 100%. InfiniteLoop bugs are hardest — require reasoning about loop termination.

Comparison with Published QuixBugs Results

Method	Type	Bugs Fixed
GenProg	Search-based APR	4/40
TBar	Template-based APR	14/40
CoCoNuT	Neural MT	19/40
ChatGPT	Single-prompt LLM	31/40
Baseline (Ours)	Single-prompt LLM	23/31
Multi-Agent (Ours)	Agentic LLM	24/31

Case Study: bitcount

`n ^= n - 1` # bug: should be `&=`

- 1. Detection:** all 5 tests time out → InfiniteLoop
- 2. Localization:** line 4 flagged
- 3. Debugging:** XOR creates a bitmask, not a bit-clear → loop never terminates

4. Patch Gen: `n &= n - 1`

5. Apply: compiles cleanly

6. Validate: all 5 tests pass

Result: fixed on attempt 1, 55.1s.

HumanEvalFix Results (164 problems)

Backend	System	Fixed	Rate	Pass@1
Gemini	Multi-Agent	112/164	68.3%	62.8%
	Baseline	109/164	66.5%	66.5%
Local (Qwen 7B)	Multi-Agent	89/164	54.3%	42.7%
	Baseline	87/164	53.0%	53.0%

Multi-agent leads on *both* backends (+1.8 / +1.3 pts). Pass@1 $\ll$ final rate $\Rightarrow$ retry loop doing real work here.

DebugBench Results — Where Baseline Wins

Backend	System	Fixed	Rate	Pass@1
Gemini	Multi-Agent	93/119	78.2%	73.9%
	Baseline	112/119	94.1%	94.1%
Local (Qwen 7B)	Multi-Agent	70/119	58.8%	49.6%
	Baseline	70/119	58.8%	58.8%

Diagnosed confound: 9/26 multi-agent failures show quota-rotation noise in the log. Corrected estimate: **84.5%** (gap narrows from 15.9 to ~9.6 pts, doesn't vanish). DebugBench's bugs are small & mechanical — extra reasoning rounds tend to *overturn* an already-correct first read.

QuixBugs on the Local Backend

System	Fixed	Rate	Pass@1	Avg. Time
Multi-Agent	16/31	51.6%	35.5%	38.5s
Baseline	18/31	58.1%	58.1%	2.1s

Baseline wins by 6.5 pts — the *opposite* direction from the Gemini result on the same benchmark. Weaker model's first-pass reasoning gets worse, not better, with more scaffolding context.

Cross-Backend Synthesis

Dataset	Local MA	Gemini MA	Local Base	Gemini Base
QuixBugs	51.6%	77.4%	58.1%	74.2%
HumanEvalFix	54.3%	68.3%	53.0%	66.5%
DebugBench	58.8%	78.2%	58.8%	94.1%

Central Finding

Multi-agent's benefit over baseline is **jointly conditional** on backend capability *and* bug structure — not a fixed property of the architecture. Stronger backend $\Rightarrow$ higher absolute performance for both systems, but *not* a uniformly wider multi-agent lead.

When Multi-Agent Helps

- **Structured reasoning:** targeted fixes when the root cause isn't obvious from the buggy code alone (e.g. `bitcount`).
- **Iterative refinement:** retry-with-feedback recovers failed attempts (`levenshtein`; `HumanEvalFix Pass@1` gap).
- **Modularity:** swapped Gemini ↔ local Qwen with zero changes to agent logic or prompts.
- **Transparency:** interpretable log (`classify` → `localize` → `diagnose` → `fix`) aids trust and failure analysis.

When It Doesn't

- **Error propagation with a weaker model:** on the local backend, multi-agent's Pass@1 trails baseline on *every* benchmark — a wrong intermediate diagnosis misleads downstream patch generation.
- **Unproductive re-litigation on mechanical bugs:** on DebugBench, extra reasoning rounds more often *overturn* a correct first read than improve it (baseline: 94.1% single-shot).

Time–accuracy tradeoff: multi-agent is 5–20× slower per problem; justified only where it's actually more accurate (QuixBugs/Gemini, HumanEvalFix). Unfavorable on both axes for DebugBench and the local backend.

Limitations & Threats to Validity

- Scope limited to isolated, single-function/algorithmic bugs — not multi-file, repo-scale changes.
- Recovery quality depends on test-case coverage/quality.
- DebugBench judged only against 1–3 public examples (no hidden suite); canonical solutions pass at 86% — an approximate fidelity ceiling.
- DebugBench/Gemini result confounded by quota-rotation noise (raw vs. corrected reported).
- Single run per cell (no repeated-seed averaging); LLM outputs are non-deterministic even at low temperature.
- DebugBench evaluated on a stratified sample (119/1,246), not the full pool.

- Multi-agent recovery beats single-prompt baseline on QuixBugs/Gemini (77.4% vs. 74.2%) and HumanEvalFix (both backends).
- Baseline matches or beats multi-agent on the weaker local backend (all 3 benchmarks) and on DebugBench/Gemini (94.1% vs. 78.2%, $\sim 84.5\%$ corrected).
- **Central conclusion:** the architecture's value is *jointly conditional* on backend capability and bug structure — not a universal win or loss.
- Three infrastructure defects found only via multi-hour unattended runs, diagnosed and fixed — of independent methodological value.

Future Work

- Extend to more languages (Java, C++, JS) — all three benchmarks already provide parallel data.
- Re-run DebugBench/Gemini under a paid tier / more keys to eliminate quota-noise confound.
- Evaluate a wider range of backend capabilities (mid-sized open models).
- Extend to repository-scale benchmarks: Defects4J, BugsInPy, SWE-bench.
- Adaptive controller: route a bug to multi-agent or single-prompt based on a cheap complexity estimate.
- Real CI/CD integration (e.g., GitHub Actions); debate/consensus agent patterns; static-analysis-assisted localization.

Thank You

Questions?